AI Image Generator

5. Module Reference

Version 2.2
22 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Images, video and music on one GPU: Z-Image Turbo, SDXL fine-tunes, Qwen-Image, FLUX.1, LoRAs, LTX-Video, Wan 2.1, MusicGen and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and an Admin panel that manages models, LoRAs and downloads and shows live system statistics.

5. Module Reference

5.1 app.py (about 640 lines)

Entry point of the web application: the Studio and History tabs, generation, and the mounting of the Admin panel.

Name	Kind	Description
OUTPUT_DIR, UPSCALE_CHOICES, HELP, EMPTY_STATE, THUMB_DIR	constants	Outputs path, the 2K/4K/8K choices, the prompt help table, the empty-gallery message and the thumbnail cache folder.
MANAGER, STARTUP_MODEL, UPSCALER, STATIC_GPU	globals	The ModelManager, the start-up model (FRAZILE_STARTUP_MODEL, else the registry setting, "none" to skip), the Upscaler and static GPU facts, created once at import. applog.install() runs first so every later print reaches the Logs tab.
ui_lora_weight(stem), default_negative()	functions	Registry-backed defaults: a LoRA's own weight (else the global setting) and the default negative prompt.
pick_model(settings, choice)	function	Resolves the model: prompt directive, then dropdown, then (Auto) a model of the family the requested LoRAs were trained for, then style-based choice among SDXL specialists. Returns (spec, note).
add_ui_loras(), trigger_hints(), preview()	functions	Merge ticked LoRAs into the settings, remind of missing trigger words, and render the live "Will generate" line (including whether a load or download is coming).
details_markdown(rec), upscale_record(rec, target_px, exact_size)	functions	The Markdown report for one record; upscaling of a record's native PNG.
generate(prompt, negative, model_choice, loras_ui, progress)	function	Runs the full pipeline under the ModelManager lock for all requested images; returns gallery, report, records, selection, Upscale bar, hint, loaded line and empty-state visibility.
clear_all(), upscale_hint(), on_select(), upscale()	functions	Clear button, the hint under the Upscale bar, gallery selection tracking, and the Upscale bar handler.
toggle_chip(), chip_seed(), reuse_seed(), drop_seed()	functions	Quick-hint chips (a hint in the same group replaces its siblings), the random-seed chip, Reuse this seed and New variation.
thumbnail_for(), history_rows(), history_table(), history_gallery(), history_stats(), refresh_history(), show_record(), select_history(), select_history_thumb(), reuse_prompt(), delete_history()	functions	History tab: 384 px JPEG thumbnails cached in outputs/.thumbs/, filters (search, model, upscaled only, limit), table and grid views, details, reuse prompt with or without seed (switches to the Studio tab) and deletion.
refresh_header()	function	The header with loaded model, VRAM, temperature, GPU load and running downloads, refreshed by a 5 s timer.

The UI is a gr.Blocks with fill_width=True and three tabs (Studio, History, Admin). GPU events (generate, upscale, admin load and unload) share the concurrency id "gpu" so they queue behind each other; statistics and timers use "stats" with a limit of three so they keep refreshing during a generation. launch() reads GRADIO_SERVER_NAME, GRADIO_SERVER_PORT and FRAZILE_NO_BROWSER, allows 8 GB uploads and passes the theme and CSS.

5.2 models.py (about 680 lines)

Name	Kind	Description
ModelSpec	dataclass	key, name, source (HF repo or path), family (sdxl, zimage, qwen or flux), steps, guidance, scheduler, variant, negative, aliases, styles, description, download_gb, local.
BUILTIN	constant	The nine built-in specs with their defaults and recommended samplers.

Name	Kind	Description
local_checkpoints(), all_models(include_disabled), family_models(family), model_words()	functions	Scan models/checkpoints/; the merged list (built-ins with registry overrides, local checkpoints, admin-added models, minus disabled ones); the on-disk models of one family in preferred order; every key and alias for the parser.
resolve(name)	function	Finds a model by key, alias, name, repo id or path, including disabled ones; raises KeyError if unknown.
model_for_style(style), is_downloaded(spec), on_disk_bytes(spec), hf_cache_dir(repo), display_name(source), spec_for_source(source)	functions	Auto-selection lookup, on-disk check and size (file, HF cache blobs, converted caches), cache folder of a repo, and source-to-spec mapping for the UI.
find_lora(name), available_loras(family), lora_info(name)	functions	Lookup by file stem or registry display name; the enabled LoRA stems, optionally of one family; a LoRA's registry metadata.
a1111_to_compel(prompt)	function	Rewrites (word:1.3) into compel's (word)1.3.
ModelManager	class	Holds one pipeline and a re-entrant lock. load() dispatches to the SDXL, Z-Image, Qwen-Image or FLUX loader; unload() frees VRAM; apply_loras() skips LoRAs of another family, loads once and activates with weights on top of base adapters; encode() returns weighted embeddings via compel; release_cache(); generation_kwargs() adds family-specific arguments.
STARTER_LORAS, fetch_starter_loras()	constant, function	The two documented LoRAs and the `models.py fetch-loras` command.

Running the module directly lists every model, its family and whether it is downloaded.

5.3 registry.py (about 300 lines)

Name	Kind	Description
REGISTRY_PATH, FAMILIES, SCHEDULERS, FAMILY_PREFERRED, DEFAULT_SETTINGS, OVERRIDABLE	constants	models/registry.json; the four families with descriptions; sampler names; which models to prefer per family; default settings (startup_model, ui_lora_weight, default_negative, civitai_token, auto_release_vram, outputs_keep_days); the ModelSpec fields an admin may override on a built-in.
load(), save()	functions	Cached read (re-read when the file changes) and atomic write with an updated_at stamp.
setting(key), set_settings(key=value ...)	functions	Settings access.
apply_overrides(spec), custom_models(), model_entry(), is_enabled(), is_custom(), upsert_model(key, custom, key=value ...), remove_model(key), set_enabled()	functions	Model entries: overrides for built-ins and local files, full specs for admin-added models, enable flag, create or update, delete or reset.
lora_entries(include_missing), lora(stem), set_lora(stem, key=value ...), forget_lora(stem)	functions	LoRA records: files in models/loras/ merged with their metadata (name, family, trigger, weight, description, source, enabled, tags, size, missing flag).
summary()	function	Counts for the Admin Overview.

5.4 downloads.py (about 480 lines)

Name	Kind	Description
Job, JobManager, JOBS	dataclass, class, global	A download with status, bytes done and total, speed, ETA, message, log and a cancel flag; a two-worker thread pool that runs job functions and keeps the list; the shared instance.

Name	Kind	Description
inspect_source(text)	function	Classifies a source as hf_repo, hf_file, civitai, url or local and gathers what it would give: pipeline class and family (from model_index.json), fp16 variant, sizes, gated flag, Civitai type, base model and trigger words, warnings. Never raises.
download_file(info, as_type, rename)	function	Job function: streams one .safetensors file into models/checkpoints/ or models/loras/ with resume (Range), progress, Hugging Face and Civitai tokens, and refuses HTML answers.
download_repo(spec)	function	Job function: snapshot_download of a diffusers repo with ignore patterns per family (fp16 only for SDXL, no bin or onnx, no Qwen text encoder) and byte progress read from the cache folder; fetches the Qwen 4-bit text encoder and Lightning LoRA too.
delete_model_files(spec)	function	Removes a local file or the repo's HF cache folder and the converted caches under models/cache/.
hf_token(), civitai_token(), safe_stem(), unique_path(), human_bytes()	functions	Helpers.

Running `python downloads.py <source>` prints the inspection result as JSON.

5.5 admin_ui.py (about 600 lines)

Name	Kind	Description
build_admin(manager, upscaler, static_gpu, prompt_box, tabs, model_choice, lora_choice, loaded_md)	function	Builds the eight sub-tabs inside the Admin tab and wires their events; receives the Studio's prompt box, tab bar and dropdowns so Admin actions can update them.
guard(unlocked, confirm), unlock()	functions	Confirmation box and optional FRAZILE_ADMIN_PASSWORD check for destructive actions.
overview_refresh()	function	System tiles, activity tiles and the per-model statistics table.
models_df(), select_model(), model_editor_values(), save_model(), load_model(), unload_model(), download_model(), set_startup(), reset_or_remove_model(), delete_model_files()	functions	Models tab.
loras_df(), select_lora(), lora_editor_values(), save_lora(), try_lora(), delete_lora()	functions	LoRAs tab; try_lora fills a test prompt and switches to the Studio.
check_source(), add_source(), upload_file()	functions	Add tab: inspection with form pre-fill, registration plus background download, browser upload.
downloads_refresh(), cancel_job(), clear_jobs()	functions	Downloads tab.
system_refresh(check_internet)	function	The nine tiles and six sections of the System tab.
settings_values(), save_settings(), hf_status_html(), save_hf_token(), cleanup_outputs()	functions	Settings tab.
logs_text(n, contains, level)	function	Logs tab.

5.6 ui_theme.py, ui_common.py, applog.py

Module	Contents
ui_theme.py (about 190 lines)	Colour constants, build_theme() (Frazile AI palette applied to light and dark mode alike), CSS for header, chips, cards, badges, tiles, meters, jobs and logs, and the badge helpers.
ui_common.py (about 170 lines)	model_choices() and lora_choices() for the Studio dropdowns with update helpers; header_html() with live pills; loaded_line(); model_cards_html(); tile(), kv() and jobs_html().
applog.py (about 120 lines)	install() tees stdout and stderr into a 3,000-line ring buffer (keeping only the final state of progress bars; lines mentioning error, exception, traceback or failed count as errors); log(), tail(), tail_text(), counts().

5.7 prompt_parser.py (315 lines)

Name	Kind	Description
Settings	dataclass	prompt, width, height, steps, guidance, count, seed, style, upscale_to, exact_size, notes, model, loras, steps_explicit, guidance_explicit, quality, fidelity. Methods: summary(), upscaled_size(), apply_defaults(base_steps, base_guidance).
RATIO_BUCKETS, ORIENTATION_WORDS, STYLE_SUFFIXES, UPSCALE_TARGETS	constants	As in v1.0: eleven SDXL buckets, orientation regexes, twelve style presets, upscale words.
parse(text)	function	Returns a Settings. Order: LoRA tags, model directive, explicit resolution, ratio, orientation, upscale words, steps, guidance, count, seed, style, tidy-up, then apply_defaults with the SDXL baseline.

Quality and fidelity words no longer set fixed numbers; they are recorded and applied relative to the chosen model's defaults. Running the module directly prints the interpretation of twelve sample prompts.

5.8 sysinfo.py (358 lines)

Name	Kind	Description
human_bytes(), human_duration()	functions	Formatting helpers used by the Admin panel.
cpu()	function	Processor name (registry on Windows, /proc/cpuinfo on Linux), cores, frequency, usage per core, load averages, process and thread counts (cached 30 s), boot time, uptime.
memory(), disks()	functions	RAM and swap; partitions with usage, disk I/O totals, and the sizes of outputs/, models/, the HF cache and the database (cached 60 s).
network(), internet()	functions	Hostname, primary IP, totals, per-second rates from the previous sample, errors and drops, connections and listening ports, every interface; optional TCP reachability of three hosts.
gpu(), power()	functions	gpu_info static and live data plus PCIe link, VBIOS, max clocks, power limits, throttle reasons and GPU processes from nvidia-smi; power draw against limit, Windows power plan, battery.
software(), app(), everything()	functions	Library and driver versions and the Docker flag; the app process; one call that collects all of the above.

5.9 upscaler.py (109 lines)

Name	Kind	Description
MODEL_PATH, TARGETS, TILE, OVERLAP	constants	models/RealESRGAN_x4plus.pth; 2k 2560, 4k 3840, uhd 3840, 8k 7680; 512 and 32 pixels.
Upscaler(device, model_path)	class	Loads the model with spandrel, switches it to half precision on CUDA, exposes name and scale.
Upscaler.upscale_once(img)	method	One x4 pass over padded tiles under inference_mode.

Name	Kind	Description
Upscaler.upscale_to(img, long_side)	method	Repeats passes until the long side reaches the target, resizing before a second pass, then Lanczos-resamples to exactly long_side. Returns (image, info).

5.10 gpu_info.py (97 lines)

Name	Kind	Description
snapshot()	function	Runs nvidia-smi once and returns temperature_c, power_w, power_limit_w, gpu_util_pct, mem_util_pct, mem_used_mb, mem_total_mb, sm_clock_mhz, mem_clock_mhz, fan_pct, driver_version, pstate. Returns {} if nvidia-smi fails.
static_info()	function	gpu_name, vram_total_gb, compute_capability, multiprocessors, cuda_version, cudnn_version, torch_version, diffusers_version, python_version, os.
torch_memory()	function	vram_allocated_gb, vram_reserved_gb, vram_peak_gb from the torch allocator.

5.11 db.py (about 270 lines)

Name	Kind	Description
DB_PATH, SCHEMA, NEW_COLUMNS	constants	generations.db next to the module; CREATE TABLE plus index; columns added after the first release.
connect()	function	Opens a connection with Row factory, applies schema and migrations.
insert(record)	function	Inserts known keys as columns; unknown keys are packed into extra_json. Returns the row id.
update(row_id, fields)	function	New in v2.0: sets known columns on one row; used by the Upscale bar.
fetch(limit, search), fetch_filtered(limit, search, model_id, upscaled_only), get(row_id), stats()	functions	Newest-first list with LIKE search, the filtered variant used by History, one record, aggregates.
delete(row_id, remove_files), stats_by_model(), recent_counts(), lora_usage(), models_used()	functions	New in v2.1: delete a record and optionally its files; per-model images, timings, VRAM and last use; images today, this week and GPU seconds in 24 h; LoRA use counts from the parser notes; distinct models for the History filter.

5.12 generate.py (125 lines)

Standalone command-line generator. Arguments: prompt, --negative, --steps, --guidance, --width, --height, --seed, --count, --model (key, repo id or path), --lora NAME[:WEIGHT] (repeatable), --out, --list-models. Uses the same ModelManager, so weighting, samplers and LoRAs behave exactly as in the app. It does not write to the database.

5.14 media.py (about 330 lines)

The video and music engine. PROFILES describes each architecture (ltx, wan, musicgen, stable_audio): kind, fps and frame-count rule, maximum length, default steps and guidance, negative prompt, VRAM need and whether image-to-video is supported. arch_of() reads the repo's model_index.json or config.json (cached) so admin-added repos get the right loader; profile_for() merges the registry's steps and guidance; repo_allow_patterns() limits a download to the diffusers sub-folders (the LTX repo carries 250 GB of single-file checkpoints at its root) or to the transformers files of MusicGen; frames_for() rounds a length in seconds to 8k+1 (LTX) or 4k+1 (Wan) frames.

- MediaManager(lock): one loaded video or music model. load() dispatches to _load_ltx, _load_wan, _load_musicgen or _load_stable_audio; each diffusers part is loaded with device_map onto the GPU shard by shard, because the fp32 text encoders (19 to 23 GB on disk) do not fit the machine's free RAM.

_text_encoder() converts to bf16 on the first load and saves a copy under models/cache/<key>-text_encoder-bf16/ for later loads, and can quantise to 8-bit with bitsandbytes (Wan on 24 GB cards).

- generate_video(prompt, negative, width, height, frames, steps, guidance, seed, image, on_step): returns PIL frames. LTX uses frame_rate 24, decode_timestep 0.03 and decode_noise_scale 0.025, VAE tiling, and an LTXImageToVideoPipeline built from the same components when a start image is given. Wan uses the UniPC scheduler with flow_shift 3.0 (480p) and an fp32 VAE.
- generate_music(prompt, negative, seconds, guidance, temperature, seed, on_progress): returns (samples, sample_rate). MusicGen runs in fp16 with do_sample, guidance_scale and max_new_tokens = seconds x 50; a StoppingCriteria reports token progress. Stable Audio calls the diffusers pipeline with audio_end_in_s.
- prepare_for(spec, image_manager, media_manager): the GPU rule. Loading an image model unloads a video model (and a music model if the image model is one of the big flow models); loading a video model unloads the image model; loading a music model unloads the image model only when free VRAM is below the music model's need plus 2 GB.

5.15 media_ui.py (about 420 lines)

- generate_video(): resolves the model, size, frame count and seed, crops the optional start image, takes the GPU lock, loads through prepare_for(), generates with step progress, encodes the MP4 with video_io.write_mp4, records the row (kind video, width, height, frames, fps, duration_s plus the usual telemetry) and returns the player value, the details report, a state dict and the upscale bar.
- upscale_video(state, target_px): reads the current clip frame by frame, runs the compact video upscaler (video_upscaler(): realesr-general-x4v3 through the same Upscaler class, one x4 pass plus a resample) on each frame and streams the result into a new MP4 (crf 18), keeping the sound track if one was added; updates the upscale columns of the record.
- add_soundtrack(state, track): muxes a track from the Music tab under the current clip with a one-second fade-out; stored in soundtrack_path.
- generate_music(): the same shape for tracks (kind music, duration_s, sample_rate), written as 16-bit WAV.
- video_defaults() / music_defaults(): fill steps, guidance, negative prompt, maximum length and the start-image box when the model box changes. build_video_tab() and build_music_tab() lay the tabs out and wire the events; refresh_*_choices() update the model and track boxes when a tab is opened.

5.16 video_io.py (about 190 lines)

- write_mp4(frames, path, fps, crf, audio): pipes raw RGB frames into the bundled ffmpeg (libx264, yuv420p, faststart) so a 4K clip is never held in memory; read_frames(path) yields PIL frames with the clip's metadata; probe(), video_thumbnail(), mux_audio().
- write_wav(samples, sample_rate, path) through soundfile (peak-normalised when needed); write_mp3(); waveform_png() draws the History thumbnail with the studio's pink-to-yellow gradient; audio_seconds().

5.13 Supporting files

File	Purpose
requirements.txt	Python dependencies with the note on installing the CUDA PyTorch build first
run-app.bat	Double-click launcher that runs app.py inside the virtual environment
README.md	User-facing quick start, model table, prompt reference, Admin and Docker sections
Dockerfile, docker-compose.yml, .dockerignore	Container image and service definition (section 12)
models/RealESRGAN_x4plus.pth	Upscaler weights for images

File	Purpose
models/realesr-general-x4v3.pth	Compact Real-ESRGAN weights for video frames (4.7 MB, fetched on the first video upscale)
models/checkpoints/, models/loras/	Checkpoints and LoRAs, filled by the Admin panel or by hand
models/registry.json	Admin registry: settings, model overrides and additions, LoRA metadata (created on first edit, not in git)
outputs/.thumbs/	Thumbnail cache for the History grid (JPEG for images and first frames of videos, PNG waveforms for tracks)
models/cache/<key>-text_encoder-bf16/	bf16 copies of the video models' text encoders, written on the first load (not in git)
docs/	This documentation set as PDFs
docs-src/build_pdf.py, docs-src/images/	Source of this documentation and its figures